Strategy

# Strategy Pattern

Swap the algorithm at runtime

DEFINITION

Define a family of algorithms, encapsulate each one, and make them interchangeable.

Strategy encapsulates each algorithm of a family behind a common interface so they are interchangeable at one call site. The Context holds a strategy and delegates to it; behavior is injected by composition, not baked in by inheritance, and the client chooses which algorithm — the strategy never swaps itself.

WHY IT MATTERS

When one task has many variants, packing them into branching conditionals makes the class grow with every new case and tangles unrelated algorithms together. Strategy pulls each into its own unit: you add a variant without editing the context (OCP), test algorithms in isolation, and swap them at runtime. The cost — the client must know enough to choose, and a fat interface can pass data some strategies ignore.



Vasyl Krupka · Ukraine

## How it works

| | |
|---|---|
| **Strategy** | The common interface every algorithm implements. |
| **ConcreteStrategy** | One self-contained algorithm behind that interface. |
| **Context** | Holds a strategy and delegates the work to it. |

### HOW TO APPLY

1. Find the behavior that varies and the branches that select it.
2. Define one Strategy interface (in JS, often just a function signature).
3. Make each branch a ConcreteStrategy (or a function) implementing it.
4. Inject the chosen strategy into the Context; let the caller pick at runtime.

> LITMUS TEST
>
> Does this task have several interchangeable algorithms selected at runtime, or an if/else ladder picking behavior? If yes, make each branch a strategy.



Vasyl Krupka · Ukraine

## Smell → fix

A Sorter hard-codes every ordering in one branching method:

```
class Sorter {
sort(d: number[], mode: string) {
if (mode === 'asc') return d.sort((a,b) => a-b)
if (mode === 'desc') return d.sort((a,b) => b-a)
// a new order edits this method
}
}
```

↓ EXTRACT EACH ALGORITHM BEHIND ONE INTERFACE

```
type Sort = (d: number[]) => number[]
const asc: Sort = d => [...d].sort((a,b) => a-b)
const desc: Sort = d => [...d].sort((a,b) => b-a)
function sortBy(d: number[], s: Sort) {
return s(d) // client picks the algorithm
}
```

Each ordering is now an independent value the caller passes in; sortBy delegates to it. A new order is a new function — sortBy never changes (OCP), and you can test each algorithm alone.

USE WHEN

- ✓ Pluggable behavior (sort, pricing, auth, compression)
- ✓ Replace an if/else ladder

AVOID WHEN

- ▲ Behavior never varies — needless indirection

# — Trade-offs & pitfalls

PROS

+ OCP
+ Test algorithms in isolation
+ Swap at runtime

CONS

− Client must know the strategies
− More objects

COMMON PITFALLS

▲ The client must understand the options well enough to choose the right strategy.

▲ A fat Strategy interface forces every algorithm to accept data some never use — keep it minimal.

▲ Reaching for classes when a closure would do: in JS a function is a fine strategy; classes earn their keep only when an algorithm carries state or config.

WHERE YOU SEE IT

- Array sort comparators; Passport.js auth strategies (local, OAuth, JWT).
- Pluggable pricing, validation, or a compression codec (gzip / brotli) behind one interface.
- Replacing an if/else ladder over "type" with a registry of handler functions.

SEE ALSO

| | |
|---|---|
| **State** | identical UML — but State self-transitions, while a Strategy is client-set and stays put |
| **Bridge** | both favor composition; Bridge varies an abstraction and its implementation on two axes |
| **OCP** | swapping strategies is how you extend behavior without editing the context |
| **Polymorphism** | the GRASP principle Strategy applies — let the type pick the behavior |

# Interview & sources

### Strategy vs State?

Identical UML; a Strategy is chosen by the client and stays put, State objects swap themselves as internal conditions change.

### Strategy vs Template Method?

Strategy varies a whole algorithm via composition and runtime swapping; Template Method varies steps via subclass inheritance.

### Do I need a class per strategy?

No — in JS a function is a perfectly good strategy; use classes when an algorithm carries its own state or configuration.

### Are strategies stateless?

Usually yes, which makes them safe to share as singletons; if one holds state, do not reuse the instance across contexts.

REMEMBER

A family of interchangeable algorithms behind one interface — the client picks; the algorithm varies independently.

SOURCES

GoF — "Design Patterns" (1994): Strategy (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Strategy